

KONSPEKT ZAJĘĆ LABORATORYJNYCH

LABORATORIUM 10

Animacje i micro-interactions

Przedmiot	Zaawansowany interfejs użytkownika
Prowadzący	mgr inż. Dominik Aksamit
Rok / Semestr	I rok, semestr 1, studia stacjonarne
Czas trwania	90 minut (2 × 45 min)

1. Cele zajęć

Po ukończeniu zajęć student:

- rozróżnia techniki animacji (CSS transitions, CSS animations, JS animations, Web Animations API, Framer Motion) i potrafi dobrać odpowiednią do kontekstu
- implementuje efekty hover i transition dla nawigacji z użyciem czystego CSS
- tworzy page transitions z komponentem AnimatePresence (Framer Motion v10+)
- implementuje animacje list z efektem stagger (staggerChildren) oraz drag & drop z useDragControls
- buduje skeleton loading animations z shimmer effect (CSS @keyframes)
- tworzy toast notifications z animowanym wejściem i wyjściem
- stosuje media query prefers-reduced-motion i uzasadnia jej znaczenie dla dostępności
- ocenia, które animacje poprawiają doświadczenie użytkownika, a które je pogarszają

Powiązane efekty kształcenia

- **Wiedza:** IN2_W01 (technologie UI), IN2_W03 (zasady projektowania), IN2_W04 (testowanie UI)
- **Umiejętności:** IN2_U01 (UCD), IN2_U02 (dane o zachowaniach), IN2_U04 (integracja z API)
- **Kompetencje:** IN2_K01 (etyka i dostępność), IN2_K03 (narzędzia prototypowania)

2. Harmonogram zajęć

Czas	Blok	Zawartość
0–10 min	Wstęp	Przegląd celów, weryfikacja środowiska, kontynuacja projektu React z poprzednich zajęć
10–25 min	Teoria	Przegląd technik animacji — wykład + live demo w DevTools (panel Animations)
25–30 min	Setup	Instalacja Framer Motion v10+, hello-world animacji razem z prowadzącym
30–40 min	Etap A	CSS hover na kartach filmów + transition dla nawigacji
40–50 min	Etap B	Page transitions z AnimatePresence + klucze routera
50–60 min	Etap C	Stagger list animations + drag & drop z Reorder.Group

60–70 min	Etap D	Skeleton loading (CSS shimmer) + Toast notifications z AnimatePresence
70–80 min	Dostępność	prefers-reduced-motion — teoria + wdrożenie w projekcie (min. 2 miejsca)
80–90 min	Podsumowanie	Code review, omówienie błędów, pytania do kolokwium, kryteria oceny

3. Przegląd technik animacji (15 min)

Zajęcia zaczynają się od krótkiego (ok. 12 min) wykładu z demonstracją na żywo. Prowadzący otwiera Chrome DevTools → [More tools](#) → [Animations](#) i pokazuje działanie każdej techniki na przykładzie prostego elementu. Dokumentacja: <https://developer.chrome.com/docs/devtools/css/animations>

Technika	Przypadek użycia	Zalety	Wady	Wydajność
CSS Transitions	Hover, focus, active — prosta interpolacja między 2 stanami	Deklaratywne, GPU-friendly, zero JS	Brak kontroli keyframes, brak sekwencji	★★★★★
CSS Animations	Skeleton loaders, pulse, dekoracje	Pełne keyframes, bez JS, wydajne	Trudna synchronizacja z logiką app	★★★★★
JS / rAF	Fizyczne symulacje, canvas, gry	Pełna kontrola, dynamika w runtime	Dużo kodu, ryzyko jank	★★★★☆
WAAPI	Natywne API — <code>el.animate()</code> , lepsza kontrola niż CSS	Natywne, JavaScript API, brak deps	Niższe wsparcie przeglądarek (IE)	★★★★★
Framer Motion	React — page transitions, listy, drag&drop, layout anim.	Deklaratywne, variants, spring physics, layout anim.	Zależność ~30kB, tylko React	★★★★★

Kluczowa zasada: Animuj wyłącznie właściwości composite: transform i opacity. Animacja width, height, top, left wyzwała layout/paint i powoduje jank (spadki FPS). Sprawdzaj w DevTools → Performance → Rendering.

4. Instalacja i podstawy Framer Motion v10+

4.1 Instalacja

```
# Instalacja w projekcie React
npm install framer-motion

# Weryfikacja wersji
npm list framer-motion
```

4.2 Import — najważniejsze eksporty

```
import { motion, AnimatePresence, useReducedMotion } from 'framer-motion';
import { Reorder } from 'framer-motion'; // drag & drop listy
```

4.3 Podstawowy komponent motion

```
// Zamień dowolny element HTML na motion.*
<motion.div
  initial={{ opacity: 0, y: 20 }} // stan startowy
  animate={{ opacity: 1, y: 0 }} // stan docelowy
  exit={{ opacity: 0, y: -20 }} // stan przy usuwaniu z DOM
  transition={{ duration: 0.3, ease: 'easeOut' }}
/>
```

```
>
  Treść
</motion.div>
```

4.4 Variants — reużywalne definicje animacji

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: { staggerChildren: 0.08, delayChildren: 0.1 },
  },
};

const itemVariants = {
  hidden: { opacity: 0, y: 16 },
  visible: { opacity: 1, y: 0, transition: { duration: 0.25 } },
};
```

5. Zadanie: animacje w projekcie Movie Browser

Studenci otwierają projekt React z poprzednich zajęć. Każdy etap to osobny commit.

Etap A — CSS hover & transitions (10 min)

- **Cel:** dodanie efektów stanu do kart filmów i linków nawigacji
- `transform: scale(1.03) + box-shadow` przy hover na karcie
- Transition czasu ładowania strony w navbarze: `transition: background 0.2s ease`
- `will-change: transform` na elementach często animowanych (z umiarem!)
- Focus-visible ring — dostępność klawiatury: `:focus-visible { outline: 2px solid #0043FF }`

```
/* MovieCard.css — hover efekt */
.card {
  transition: transform 0.2s ease, box-shadow 0.2s ease;
  will-change: transform;
}

.card:hover {
  transform: scale(1.03) translateY(-2px);
  box-shadow: 0 8px 24px rgba(0, 67, 255, 0.15);
}

.card:focus-visible {
  outline: 2px solid #0043FF;
  outline-offset: 2px;
}
```

Etap B — Page transitions z AnimatePresence (10 min)

- **AnimatePresence** — pozwala animować elementy usuwane z drzewa React (exit animations)
- `mode="wait"` — czeka na zakończenie exit przed renderem nowej strony
- Klucz = ścieżka routera (`location.pathname`) — wyzwala re-mount przy zmianie trasy

```
// App.jsx — owinięcie tras w AnimatePresence
import { AnimatePresence } from 'framer-motion';
import { useLocation } from 'react-router-dom';

const pageVariants = {
  initial: { opacity: 0, x: -16 },
  animate: { opacity: 1, x: 0, transition: { duration: 0.28, ease: 'easeOut' } },
  exit: { opacity: 0, x: 16, transition: { duration: 0.18, ease: 'easeIn' } },
};
```

```
function App() {
  const location = useLocation();
  return (
    <AnimatePresence mode='wait'>
      <Routes location={location} key={location.pathname}>
        <Route path='/' element={<motion.div variants={pageVariants}
          initial='initial' animate='animate' exit='exit'>
            <HomePage /></motion.div>} />
      </Routes>
    </AnimatePresence>
  );
}
```

Etap C — Stagger & Drag-and-Drop (10 min)

- **staggerChildren** — każdy element listy pojawia się z opóźnieniem 80 ms po poprzednim
- `Reorder.Group` + `Reorder.Item` — gotowy drag & drop z animacją pozycji (layout animations)
- Klucz każdego elementu musi być unikalny i stabilny (id, nie index tablicy)

```
// MovieList.jsx - stagger + drag & drop
import { motion, Reorder } from 'framer-motion';

const container = {
  hidden: { opacity: 0 },
  visible: { opacity: 1, transition: { staggerChildren: 0.08 } },
};
const item = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0 },
};

// Lista z animacją wejścia (stagger)
<motion.ul variants={container} initial='hidden' animate='visible'>
  {movies.map(movie => (
    <motion.li key={movie.id} variants={item}>
      <MovieCard movie={movie} />
    </motion.li>
  ))}
</motion.ul>

// Ulubione - drag & drop (Reorder)
<Reorder.Group axis='y' values={favorites} onReorder={setFavorites}>
  {favorites.map(movie => (
    <Reorder.Item key={movie.id} value={movie}>
      <FavoriteCard movie={movie} />
    </Reorder.Item>
  ))}
</Reorder.Group>
```

Etap D — Skeleton Loading & Toast Notifications (10 min)

Skeleton Loading — CSS shimmer

```
/* Skeleton.css */
@keyframes shimmer {
  from { background-position: -200% 0; }
  to   { background-position: 200% 0; }
}

.skeleton {
  background: linear-gradient(
    90deg,
    #e0e0e0 25%,
    #f0f0f0 50%,
    #e0e0e0 75%
  );
}
```

```
background-size: 200% 100%;
animation: shimmer 1.4s ease-in-out infinite;
border-radius: 6px;
}

@media (prefers-reduced-motion: reduce) {
  .skeleton {
    animation: none;
    background: #e0e0e0; /* statyczny placeholder */
  }
}
```

Toast Notifications z AnimatePresence

```
// ToastContainer.jsx
import { AnimatePresence, motion } from 'framer-motion';

const toastVariants = {
  initial: { opacity: 0, x: 48, scale: 0.9 },
  animate: { opacity: 1, x: 0, scale: 1,
    transition: { type: 'spring', stiffness: 300, damping: 24 } },
  exit: { opacity: 0, x: 48, scale: 0.85,
    transition: { duration: 0.18 } },
};

export function ToastContainer({ toasts }) {
  return (
    <div style={{ position: 'fixed', bottom: 24, right: 24, zIndex: 999 }}>
      <AnimatePresence initial={false}>
        {toasts.map(t => (
          <motion.div
            key={t.id}
            layout // płynne przesunięcie pozostałych toastów
            variants={toastVariants}
            initial='initial'
            animate='animate'
            exit='exit'
            style={{ marginBottom: 8 }}
          >
            {t.message}
          </motion.div>
        ))}
      </AnimatePresence>
    </div>
  );
}
```

6. Dostępność animacji — prefers-reduced-motion

WCAG 2.1 kryterium 2.3.3 (AAA): Animacje wyzwalane przez interakcję można wyłączyć, chyba że są niezbędne do przekazania informacji. Ponadto WCAG 2.3.1 (A) zabrania flashy trwające >3 s na sekundę.

6.1 Dlaczego to ważne?

- Epilepsja fotogenna — animacje mogą wyzwoić napad (około 1/100 osób)
- Vestibular disorder (vertigo) — ruchome tła i parallax powodują fizyczne mdłości
- ADHD / zaburzenia koncentracji — nadmierne animacje rozpraszają uwagę
- Migreny — jaskrawe flashe i błyski wyzwalają ataki

6.2 Implementacja w CSS (strategia opt-out)

```
/* Strategia opt-out: animacje domyślnie włączone,
   wyłącz lub zredukuj na prośbę systemu operacyjnego */

@media (prefers-reduced-motion: reduce) {
  *,
```

```

*::before,
*::after {
  animation-duration: 0.01ms !important;
  animation-iteration-count: 1 !important;
  transition-duration: 0.01ms !important;
  scroll-behavior: auto !important;
}
}

```

6.3 Framer Motion — hook useReducedMotion

```

import { motion, useReducedMotion } from 'framer-motion';

function MovieCard({ movie }) {
  const shouldReduce = useReducedMotion();

  const variants = {
    hidden: { opacity: 0, y: shouldReduce ? 0 : 20 },
    visible: { opacity: 1, y: 0 },
  };

  return (
    <motion.div
      variants={variants}
      initial='hidden'
      animate='visible'
      // Przy reduce: tylko fade (bez ruchu Y)
    >
      {movie.title}
    </motion.div>
  );
}

```

6.4 Testowanie w przeglądarce

- **Chrome DevTools:** Rendering → Emulate CSS media feature: `prefers-reduced-motion`
- **macOS:** Preferencje systemowe → Dostępność → Wyświetlanie → Ogranicz ruch
- **Windows 11:** Ustawienia → Ułatwienia dostępu → Efekty wizualne → Animacje

7. Zasady UX — co pomaga, co szkodzi?

Animacja powinna **komunikować zmianę stanu**, nie dekorować. Oto praktyczne zasady poparte badaniami UX:

✓ Poprawia UX — stosuj	✗ Szkodzi UX — unikaj
Hover transition (150–200 ms) — natychmiastowy feedback kliku/najazdu	Animacje > 300 ms na krytycznych akcjach (kliknięcie przycisku submit)
Skeleton loading — redukuje odczuwalny czas ładowania o ~20% (badania Apptentive)	Bounce/elastic na elementach systemowych — infantylizuje interfejs biznesowy
Stagger list — prowadzi wzrok użytkownika przez nową treść	Parallax na treści głównej — dezorientacja, motion sickness
Page transition — ciągłość narracyjna, redukcja disorientacji	Nieskończone pętle bez opcji zatrzymania (WCAG 2.2.2)
Toast wejście od krawędzi — nie zasłania treści, informuje nienachalnie	Flash jasnych kolorów > 3 Hz — ryzyko napadu epilepsji (WCAG 2.3.1)
Drag & drop z visual affordance — widoczna zmiana pozycji w trakcie przeciągania	Zbyt wiele simultaneousnych efektów — rozproszenie uwagi, chaos percepcyjny

7.1 Timing — wzorzec Material Design / Apple HIG

- **Mikrointerakcje (hover, click):** 100–200 ms
- **Przejścia elementów UI:** 200–300 ms

- **Page transitions:** 250–400 ms
- **Złożone sekwencje:** maks. 500 ms — powyżej użytkownik czuje opóźnienie

7.2 Easing — kiedy używać której krzywej?

- `ease-out` (deceleracja) — elementy wchodzące do ekranu: karty, modals, toasty
- `ease-in` (akceleracja) — elementy opuszczające ekran: exit animations
- `ease-in-out` — elementy przemieszczające się w obrębie ekranu
- `spring` (Framer Motion) — elementy reagujące na interakcję (drag, resize) — fizyczna naturalna odpowiedź

8. Kryteria oceny

Kryterium	Opis i wskazówki dla oceniającego
Poprawność techniczna	Brak błędów w konsoli. Poprawne użycie API Framer Motion (variants, AnimatePresence, Reorder). Animacje działają w Chrome i Firefox.
Kompletność zadania	Zrealizowane wszystkie 4 etapy (A–D). Każdy etap w osobnym commicie (git log). Brak TODO w oddanym kodzie.
Jakość UX animacji	Timing odpowiedni (100–400 ms). Easing dobrany do kierunku ruchu. Animacje nie blokują interakcji. Brak nadmiernych efektów.
Dostępność	prefers-reduced-motion zaimplementowane w min. 2 miejscach (CSS + FM hook). Brak treści flashujących > 3 Hz. Weryfikacja w DevTools Emulate.
Jakość kodu	Czytelne nazwy variants. Brak magicznych liczb (stałe named constants). Kod nie powiela logiki. Separacja animacji od logiki biznesowej.

9. Materiały dodatkowe i literatura

9.1 Dokumentacja online

- Framer Motion — oficjalna dokumentacja: <https://www.framer.com/motion/>
- Motion for React (Framer Motion v11+): <https://motion.dev/docs/react>
- MDN — prefers-reduced-motion: <https://developer.mozilla.org/docs/Web/CSS/@media/prefers-reduced-motion>
- WCAG 2.1 — kryterium 2.3.3: <https://www.w3.org/WAI/WCAG21/Understanding/animation-from-interactions>
- Chrome DevTools — panel Animations: <https://developer.chrome.com/docs/devtools/css/animations/>

9.2 Literatura z sylabusa

- Chris Badura, UXUI. Design Zoptymalizowany, Helion 2019
- Jenifer Tidwell, Charles Brewer, Aynne Valencia-Brooks, Projektowanie interfejsów. Sprawdzone wzorce projektowe, Helion 2020
- Tom Greever, Projekt doskonały. Zadbaj o komunikację z klientem, wysoki poziom UX i zdrowy rozsądek, Helion 2022